

Punto 3.4 - Proyecciones a DTOs

Práctica

Ejercicio: Añadir DTOs y métodos de proyección al repositorio de órdenes del proyecto AceriaData. Crear DTOs para resúmenes, órdenes con planchas, órdenes con detalle y órdenes con totales. Crear un caso de uso que ejecute estas proyecciones, analice el SQL generado y compare el rendimiento de proyectar versus cargar entidades completas.

Contexto del proyecto: En el punto 3.3 se añadieron proyecciones básicas con Select y tipos anónimos. En este punto se profundiza en las proyecciones a DTOs, incluyendo proyecciones con constructor, colecciones de navegación, DTOs anidados y SelectMany. Estas proyecciones se usarán en el punto 3.5 para las consultas de agregación.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO PlanchaDto

Crear el archivo src/AceriaData.Application/Dtos/PlanchaDto.cs:

```
namespace AceriaData.Application.Dtos;

public class PlanchaDto
{
    public int Id { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class PlanchaDto → declara el DTO.

Línea 5: public int Id { get; set; } → identificador de la plancha.

Línea 6: public double Espesor { get; set; } → espesor.

Línea 7: public double Ancho { get; set; } → ancho.

Línea 8: public double Largo { get; set; } → largo.

Línea 9: public decimal Peso { get; set; } → peso.

Línea 10: public bool Activa { get; set; } → indica si está activa.

Error común: si el DTO tiene propiedades que no se proyectan, EF Core deja las propiedades con su valor por defecto. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO OrdenConPlanchasDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConPlanchasDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConPlanchasDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConPlanchasDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → estado.

Línea 8: public List Planchas { get; set; } = new(); → colección de planchas.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

| Paso 4: Crear el DTO DetalleDto

Crear el archivo src/AceriaData.Application/Dtos/DetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class DetalleDto
{
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class DetalleDto → declara el DTO.

Línea 5: public string ComposicionQuimica { get; set; } = string.Empty; → composición química.

Línea 6: public double TemperaturaColada { get; set; } → temperatura de la colada.

Línea 7: public string? Notas { get; set; } → notas opcionales.

Error común: si el DTO tiene una propiedad que no se proyecta y se accede a ella, el valor es null o por defecto. Se deben proyectar todas las propiedades necesarias.

| Paso 5: Crear el DTO OrdenConDetalleDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConDetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConDetalleDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: `public class OrdenConDetalleDto` → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public string Estado { get; set; } = string.Empty;` → estado.

Línea 8: `public DetalleDto? Detalle { get; set; }` → detalle opcional.

Error común: si el DTO no marca la propiedad `Detalle` como anulable, el compilador lanza un warning. Se debe marcar como `?` si puede ser null.

| Paso 6: Crear el DTO `OrdenConTotalesDto`

Crear el archivo `src/AceriaData.Application/Dtos/OrdenConTotalesDto.cs`:

```
namespace AceriaData.Application.Dtos;

public class OrdenConTotalesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
    public double PesoPromedio { get; set; }
}
```

Línea 1: `namespace AceriaData.Application.Dtos;` → declara el espacio de nombres.

Línea 3: `public class OrdenConTotalesDto` → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public int TotalPlanchas { get; set; }` → total de planchas.

Línea 8: `public decimal PesoTotal { get; set; }` → peso total.

Línea 9: `public double PesoPromedio { get; set; }` → peso promedio.

Error común: si el DTO usa `int` para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar `decimal` para los pesos.

| Paso 7: Añadir los métodos de proyección a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime
hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();

    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable.

Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente.

Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas.

Línea 14: List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta); → método que obtiene por rango de fechas.

Línea 15: List ObtenerPorClienteOrdenadas(string cliente); → método que obtiene por cliente ordenadas.

Línea 16: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → método que combina cliente y rango.

Línea 18: List ObtenerClientesUnicos(); → método que obtiene los clientes únicos.

Línea 19: List ObtenerResumenes(); → método que obtiene los resúmenes.

Línea 20: List ObtenerResumenesPorEstado(string estado); → método que obtiene los resúmenes por estado.

Línea 21: List ObtenerOrdenesConTotales(); → método que obtiene las órdenes con totales.

Línea 23: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.

Línea 24: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.

Línea 25: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.

Línea 27: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 28: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 8: Implementar los métodos de proyección en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList()
        }).ToList();
}

public List<OrdenConDetalleDto> ObtenerOrdenesConDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        }).ToList();
}

public List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConTotalesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            TotalPlanchas = o.Planchas.Count(),
            PesoTotal = o.Planchas.Sum(p => p.Peso),
            PesoPromedio = o.Planchas.Average(p => (double)p.Peso)
        })
}
```

```

    })
    .ToList();
}

```

Línea 1: `public List ObtenerOrdenesConPlanchas()` → declara el método que obtiene las órdenes con planchas.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.Select(o => new OrdenConPlanchasDto` → proyecta al DTO.

Línea 6: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 7: `Cliente = o.Cliente,` → asigna el cliente.

Línea 8: `Estado = o.Estado,` → asigna el estado.

Línea 9: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta la colección de planchas.

Línea 10: `Id = p.Id,` → asigna el Id.

Línea 11: `Espesor = p.Espesor,` → asigna el espesor.

Línea 12: `Ancho = p.Ancho,` → asigna el ancho.

Línea 13: `Largo = p.Largo,` → asigna el largo.

Línea 14: `Peso = p.Peso,` → asigna el peso.

Línea 15: `Activa = p.Activa` → asigna el estado activo.

Línea 16: `}).ToList()` → materializa la colección de planchas.

Línea 17: `})` → cierra la proyección.

Línea 18: `.ToList();` → materializa la consulta.

Línea 21: `public List ObtenerOrdenesConDetalle()` → declara el método que obtiene las órdenes con detalle.

Línea 23: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 24: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 25: `.Select(o => new OrdenConDetalleDto` → proyecta al DTO.

Línea 26: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 27: `Cliente = o.Cliente,` → asigna el cliente.

Línea 28: `Estado = o.Estado,` → asigna el estado.

Línea 29: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta.

Línea 30: `ComposicionQuimica = o.Detalle.ComposicionQuimica,` → asigna la composición química.

Línea 31: `TemperaturaColada = o.Detalle.TemperaturaColada,` → asigna la temperatura.

Línea 32: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 33: `}` → cierra la proyección del detalle.

Línea 34: `})` → cierra la proyección.

Línea 35: `.ToList();` → materializa la consulta.

Línea 38: `public List ObtenerOrdenesConTotalesDetallados()` → declara el método que obtiene las órdenes con totales detallados.

Línea 40: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 41: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 42: `.Select(o => new OrdenConTotalesDto` → proyecta al DTO.

Línea 43: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 44: `Cliente = o.Cliente,` → asigna el cliente.

Línea 45: `TotalPlanchas = o.Planchas.Count(),` → calcula el total de planchas.

Línea 46: `PesoTotal = o.Planchas.Sum(p => p.Peso),` → calcula el peso total.

Línea 47: `PesoPromedio = o.Planchas.Average(p => (double)p.Peso)` → calcula el peso promedio.

Línea 48: `})` → cierra la proyección.

Línea 49: `.ToList();` → materializa la consulta.

Error común: si se proyecta una colección sin `ToList`, EF Core puede no materializar la colección correctamente. Se debe llamar a `ToList` dentro de la proyección para materializar la colección.

| Paso 9: Crear el caso de uso de proyecciones a DTOs

Crear el archivo `src/AceriaData.Application/UseCases/ProyeccionesDtoUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProyeccionesDtoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES A DTOS ===");

        DemostrarOrdenesConPlanchas();
        DemostrarOrdenesConDetalle();
        DemostrarOrdenesConTotalesDetallados();
        DemostrarSqlProyeccionConPlanchas();
    }

    private void DemostrarOrdenesConPlanchas()
    {
        Console.WriteLine("\n--- Órdenes con planchas ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchas();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.Planchas.Count}");
            foreach (var plancha in orden.Planchas)
            {
                Console.WriteLine($"   Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");
            }
        }
    }

    private void DemostrarOrdenesConDetalle()
    {
        Console.WriteLine("\n--- Órdenes con detalle ---");
    }
}
```

```

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConDetalle();
        foreach (var orden in ordenes)
        {
            var detalle = orden.Detalle == null ? "Sin detalle" :
            $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Detalle:
            {detalle}");
        }
    }

    private void DemostrarOrdenesConTotalesDetallados()
    {
        Console.WriteLine("\n--- Órdenes con totales detallados ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotalesDetallados();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
            {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg | Peso promedio: {orden.PesoPromedio:F2} kg");
        }
    }

    private void DemostrarSqlProyeccionConPlanchas()
    {
        Console.WriteLine("\n--- SQL generado por una proyección con colección de navegación ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .OrderBy(o => o.NumeroOrden)
            .Select(o => new
            {
                o.NumeroOrden,
                o.Cliente,
                Planchas = o.Planchas.Select(p => new { p.Id, p.Espesor, p.Peso }).ToList()
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class ProyeccionesDtoUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== PROYECCIONES A DTOS ==="); → muestra la cabecera.

Línea 18: DemostrarOrdenesConPlanchas(); → llama al método de órdenes con planchas.

Línea 19: DemostrarOrdenesConDetalle(); → llama al método de órdenes con detalle.

Línea 20: DemostrarOrdenesConTotalesDetallados(); → llama al método de órdenes con totales detallados.

Línea 21: DemostrarSqlProyeccionConPlanchas(); → llama al método de SQL de proyección con planchas.

Línea 24: private void DemostrarOrdenesConPlanchas() → declara el método.

Línea 26: Console.WriteLine("\n--- Órdenes con planchas ---"); → muestra la cabecera.

Línea 28: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchas(); → llama al método del repositorio.

Línea 29: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 31: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.Planchas.Count}");` → muestra los datos.

Línea 32: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 34: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");` → muestra los datos de cada plancha.

Línea 38: `private void DemostrarOrdenesConDetalle()` → declara el método.

Línea 40: `Console.WriteLine("\n--- Órdenes con detalle ---");` → muestra la cabecera.

Línea 42: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConDetalle();` → llama al método del repositorio.

Línea 43: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 45: `var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";` → comprueba si el detalle existe.

Línea 46: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Detalle: {detalle}");` → muestra los datos.

Línea 50: `private void DemostrarOrdenesConTotalesDetallados()` → declara el método.

Línea 52: `Console.WriteLine("\n--- Órdenes con totales detallados ---");` → muestra la cabecera.

Línea 54: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotalesDetallados();` → llama al método del repositorio.

Línea 55: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 57: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg | Peso promedio: {orden.PesoPromedio:F2} kg");` → muestra los datos.

Línea 61: `private void DemostrarSqlProyeccionConPlanchas()` → declara el método.

Línea 63: `Console.WriteLine("\n--- SQL generado por una proyección con colección de navegación ---");` → muestra la cabecera.

Línea 65: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 66: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 67: `.Select(o => new` → proyecta a un tipo anónimo.

Línea 68: `o.NumeroOrden,` → incluye el número de orden.

Línea 69: `o.Cliente,` → incluye el cliente.

Línea 70: `Planchas = o.Planchas.Select(p => new { p.Id, p.Espesor, p.Peso }).ToList()` → proyecta la colección de planchas.

Línea 71: `});` → cierra la proyección.

Línea 73: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 74: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 10: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProyeccionesDtoUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 11: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 12: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas y detalle:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };

    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 17: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 19: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 20: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 22: `context.SaveChanges();` → inserta las planchas y el detalle.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 13: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada proyección. Se observan las órdenes con sus planchas, las órdenes con detalle, las órdenes con totales detallados y el SQL generado por la proyección con colección de navegación.

Paso 14: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROYECCIONES A DTOS ===

--- Órdenes con planchas ---
OF-2024-0001 | Constructora del Norte | Pendiente | Planchas: 2
  Plancha 1 | Espesor: 10.5 | Peso: 370.5 kg
  Plancha 2 | Espesor: 12.0 | Peso: 280.8 kg
OF-2024-0002 | Constructora del Sur | Pendiente | Planchas: 1
  Plancha 3 | Espesor: 8.0 | Peso: 125.6 kg

--- Órdenes con detalle ---
OF-2024-0001 | Constructora del Norte | Pendiente | Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C
OF-2024-0002 | Constructora del Sur | Pendiente | Detalle: Sin detalle

--- Órdenes con totales detallados ---
OF-2024-0001 | Constructora del Norte | Planchas: 2 | Peso total: 651.3 kg | Peso promedio: 325.65 kg
OF-2024-0002 | Constructora del Sur | Planchas: 1 | Peso total: 125.6 kg | Peso promedio: 125.60 kg

--- SQL generado por una proyección con colección de navegación ---
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Id], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[NumeroOrden], [o].[Id], [p].[Id]
```

La primera sección muestra las órdenes con sus planchas. La segunda sección muestra las órdenes con detalle. La tercera sección muestra las órdenes con totales detallados. La cuarta sección muestra el SQL generado por la proyección con colección de navegación.

Observaciones: el SQL generado incluye un `LEFT JOIN` con la tabla de planchas. Los filtros globales de

Soft Delete se aplican automáticamente a ambas tablas. El orden de las cláusulas es el correcto: SELECT, FROM, LEFT JOIN, WHERE, ORDER BY.

Paso 15: Diagnosticar un error común

Modificar el método ObtenerOrdenesConDetalle para acceder a una propiedad del detalle sin comprobar si existe:

```
csharp
public List<OrdenConDetalleDto> ObtenerOrdenesConDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Detalle = new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}
```

Resultado esperado: si alguna orden no tiene detalle, se produce una NullReferenceException en tiempo de ejecución porque o.Detalle es null. EF Core no puede traducir la proyección a SQL y lanza una excepción.

Solución: comprobar si o.Detalle es null antes de proyectarlo, como en la implementación original.

Errores comunes del ejercicio Error Causa Solución
 NullReferenceException en DTO anidado No se comprobó si la entidad relacionada es null Usar operador ternario antes de proyectar
 Colección no materializada Se olvidó ToList dentro de la proyección
 Añadir ToList dentro de la proyección
 Propiedad no proyectada Se accede a una propiedad que no está en la proyección
 Proyectar todas las propiedades necesarias
 GroupBy sin agregación Se proyecta una propiedad que no está en el GroupBy
 Usar la clave del grupo o funciones de agregación
 SelectMany con colección vacía La entidad principal no aparece en el resultado
 Usar DefaultIfEmpty
 Average sobre colección vacía Devuelve null
 Usar DefaultIfEmpty o comprobar el resultado

■ Reto resuelto: Proyección de órdenes con planchas y detalle combinados

Reto: Crear un DTO OrdenCompletaDto que contenga el número de orden, el cliente, el estado, la colección de planchas y el detalle. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO OrdenCompletaDto:

```
namespace AceriaData.Application.Dtos;

public class OrdenCompletaDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenCompletaDto → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public string Estado { get; set; } = string.Empty;` → estado.

Línea 8: `public List Planchas { get; set; } = new();` → colección de planchas.

Línea 9: `public DetalleDto? Detalle { get; set; }` → detalle opcional.

Paso 2: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<OrdenCompletaDto> ObtenerOrdenesCompletas();
```

Línea 1: `List ObtenerOrdenesCompletas();` → declara el método.

Paso 3: Implementar el método en `OrdenRepositorio`:

```
public List<OrdenCompletaDto> ObtenerOrdenesCompletas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenCompletaDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesCompletas();` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.Select(o => new OrdenCompletaDto` → proyecta al DTO.

Línea 6: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 7: `Cliente = o.Cliente,` → asigna el cliente.

Línea 8: `Estado = o.Estado,` → asigna el estado.

Línea 9: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta la colección de planchas.

Línea 17: `}).ToList(),` → materializa la colección de planchas.

Línea 18: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta.

Línea 24: `})` → cierra la proyección.

Línea 25: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarOrdenesCompletas()
{
    Console.WriteLine("\n--- Órdenes completas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"      Planchas: {orden.Planchas.Count}");
        var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
        Console.WriteLine($"      Detalle: {detalle}");
    }
}
```

Línea 1: private void DemostrarOrdenesCompletas() → declara el método.

Línea 3: Console.WriteLine("\n--- Órdenes completas ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas(); → llama al método del repositorio.

Línea 6: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 8: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos de la orden.

Línea 9: Console.WriteLine(\$" Planchas: {orden.Planchas.Count}"); → muestra el número de planchas.

Línea 10: var detalle = orden.Detalle == null ? "Sin detalle" : \$"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C"; → comprueba si el detalle existe.

Línea 11: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarOrdenesCompletas();
Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas y detalle.
```

Resultado esperado: las órdenes se muestran con su número, cliente, estado, número de planchas y detalle.

■ Analogía final

Las proyecciones a DTOs en una acería son como los formularios estandarizados que se usan para transferir información entre departamentos. En lugar de enviar la carpeta completa de una orden con todos sus documentos, se envía un formulario con los campos que el departamento receptor necesita. El formulario tiene un nombre, un formato fijo y se puede archivar. Proyectar a un DTO con inicializador de objeto es como rellenar un formulario campo a campo. Proyectar a un DTO con constructor es como usar un formulario preimpreso que garantiza que todos los campos estén rellenos. Proyectar una colección de navegación dentro de un DTO es como adjuntar una lista de planchas al formulario de la orden. Proyectar un DTO anidado es como incluir un formulario de detalle dentro del formulario principal. SelectMany es como aplanar varias listas en una sola. GroupBy con agregaciones es como pedir un resumen por cliente con totales. Así funcionan las proyecciones a DTOs en EF Core: se seleccionan solo los datos necesarios, se estructuran en formularios estandarizados y se transfieren entre capas de forma eficiente.

■ Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `PlanchaDto`, `OrdenConPlanchasDto`, `DetalleDto`, `OrdenConDetalleDto` y `OrdenConTotalesDto`.

Añadido los métodos de proyección a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ProyeccionesDtoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas y detalle.

Ejecutado las demostraciones de órdenes con planchas, órdenes con detalle y órdenes con totales detallados.

Analizado el SQL generado por una proyección con colección de navegación.

Diagnosticado el error de no comprobar si la entidad relacionada es null.

Creado el DTO `OrdenCompletaDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido proyecciones a DTOs al proyecto `AceriaData`, incluyendo proyecciones con inicializador de objeto, proyecciones con constructor, proyecciones de colecciones de navegación, proyecciones de DTOs anidados y el operador `SelectMany`. Se ha analizado el SQL generado por las proyecciones y se ha comprobado que EF Core genera un `LEFT JOIN` para las colecciones de navegación. En el siguiente punto se estudiarán las consultas de agregación con `Count`, `Sum`, `Average`, `Min`, `Max` y `GroupBy`.
